Hub-Aware Incremental Fraud Ring Detection on Streaming Transaction Graphs

Sanjay Kumar S
School of Computer Science and Engineering
VIT-AP University
Amaravati, India
23BCE1248@vitap.ac.in

Roshan Parveen
School of Computer Science and Engineering
VIT-AP University
Amaravati, India
23BCE1256@vitap.ac.in

***Abstract*—Fraud rings share devices, cards and network addresses and move money among themselves, which graph queries can expose. Two obstacles appear on live transaction streams. Legitimate shared infrastructure (office gateways, carrier NAT, kiosks) links many unrelated accounts, and the usual remedy, a hard degree cap, hides any ring that shares a busy attribute. Whole-graph queries re-run on a timer also cost more as the graph grows. We replace the cap with a continuous hub-aware weight with time decay, detect incrementally on the k-hop region touched by each micro-batch, and pass candidate rings through a calibrated scorer that explains every alert. On a deterministic simulator with ring-level ground truth and five evasion strategies, over 5 scenarios and 10 test seeds, the method reaches ring F1 0.73 against 0.21 for the original degree-capped Cypher rules and 0.29 for Louvain communities (paired Wilcoxon $p < 0.001$). Hub weighting extends the reachable recall on shared-attribute rings from 0.53 to 0.81, though not at higher precision everywhere, and the scorer contributes most of the precision. Incremental detection flags the same rings as full recomputation and at 100,000 users takes 36.1 ms per batch against 22,501 ms. Rings that rotate identities or act slowly evade every method we tested. All results are synthetic; we state what the public AML datasets we could use can and cannot test.**

***Index Terms*—fraud rings, property graphs, streaming graph analytics, incremental detection, hub-aware weighting, explainability**

I. Introduction

Fraud rings are groups of accounts that act together: they share devices, payment cards or network addresses, and they move money among themselves in cycles or fan-in/fan-out patterns. Property graphs make these relations explicit, and graph pattern queries are a common way to surface them [1], [2]. Two problems appear as soon as such queries run on a live stream of events.

The hub dilemma. Legitimate shared infrastructure, such as an office gateway, a carrier-grade NAT address or a store kiosk, links hundreds of unrelated accounts. A query that treats every shared attribute as evidence floods analysts with false rings. The usual remedy is a hard degree cap: attributes shared by more than c accounts are ignored. The cap removes the noise but also opens an evasion path, because a ring that shares a busy attribute becomes invisible to it.

Latency on a stream. Accounts, links and transfers arrive continuously. Re-running whole-graph queries on a timer delays detection, and the cost of each pass grows with the graph rather than with the change.

We make four contributions.

- **C1, hub-aware weighting.** We replace the degree cap with a continuous weight: each shared attribute contributes $1/\ln(1+\deg)$ with exponential time decay, and transfers inside the pair add weight. Busy hubs count for little on their own, but a group that shares a hub and also trades among itself still accumulates weight (Section III).
- **C2, incremental detection.** A micro-batch engine re-evaluates only the k-hop region touched by each batch, with fan-out limits that keep that region bounded when it reaches a hub.
- **C3, calibrated, explained alerts.** Candidate rings are scored by a calibrated classifier, and each alert carries feature contributions, an evidence subgraph and counterfactuals.
- **C4, evaluation under evasion.** A deterministic simulator with ring-level ground truth and five evasion strategies, including an attacker that adapts to the detector's flags, lets us measure each component on identical streams (paired design) with confidence intervals.

II. Related Work

Graph-based fraud detection. Surveys of graph anomaly detection [2] and of graph-based fraud detection [1] cover dense-subgraph, community and pattern-based methods. Dense-subgraph methods such as FRAUDAR [3] are explicitly designed to resist camouflage edges; they operate on static bipartite graphs, while we target a stream with ring-level alerts. [cite: dense subgraph detection on streams (e.g. incremental densest subgraph)]

Community detection. Louvain modularity optimisation [4] is a common unsupervised baseline for finding groups of colluding accounts; we use it as a baseline.

Streaming and incremental graph analytics. [cite: incremental graph pattern matching / continuous subgraph queries] [cite: Neo4j GDS or streaming graph database systems]

Graph neural networks for AML. GNNs have been applied to transaction graphs such as Elliptic [5] and AMLworld [6]. They classify transactions or accounts rather than emit

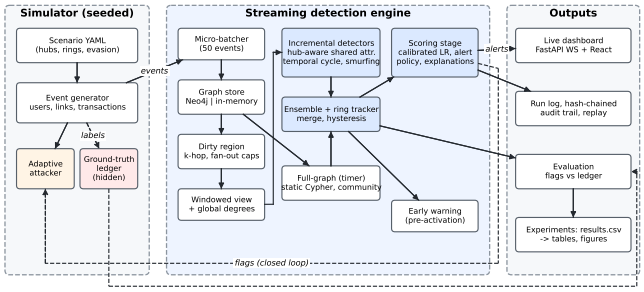


Fig. 1. Architecture. Solid arrows are the observable data path; dashed arrows carry labels (to evaluation only) and, in closed-loop runs, the detector's flags back to the adaptive attacker.

groups, and they are not evaluated here (Section VI). [cite: GNN fraud ring / camouflage-resistant GNN work]

Hub handling. Down-weighting high-degree nodes by $1/\log(\deg)$ follows the Adamic–Adar idea from link prediction. [cite: Adamic and Adar 2003, Friends and neighbors on the Web]

III. Method

Fig. 1 shows the system. A seeded simulator emits observable events (account joins and leaves, attribute links, transfers); ground truth goes only to a ledger the detector never sees.

A. Graph model

Users link to devices, cards and IP addresses (`USED_DEVICE`, `HAS_CARD`, `LOGGED_IN_FROM`) and send transfers to each other. The same contract is implemented over Neo4j (Cypher) and over an in-memory store with identical semantics; all experiments below use the in-memory store.

B. Hub-aware weighting (C1)

For two users u, v the edge weight is

$$W(u,v) = \sum_{a \in A(u) \cap A(v)} h(\deg a)\,\delta(t_a) + \min\left(\beta \sum_{x \in T(u,v)} \delta(t_x),\ \beta_{\max}\right), \tag{1}$$

where $h(d) = 1/\ln(1+d)$, $\delta(t) = 2^{-t/\tau}$ is exponential decay with half-life $\tau = 300\,\mathrm{s}$, t_a is the age of the older of the two links to attribute a, and $T(u,v)$ are the transfers between u and v in the window, with $\beta = 0.15$ and $\beta_{\max} = 0.6$. A device shared by three accounts weighs 0.72; an office address shared by fifty weighs 0.25. Candidate rings are the connected components of the graph of pairs with $W \geq \theta_e$ (and a non-zero attribute term), peeled to cores of edge density at least 0.75 and size 3–25. The edge threshold θ_e is selected on validation seeds (Section IV-B).

C. Incremental detection (C2)

Events are grouped into micro-batches of 50. For each batch the engine takes the users it touched, expands them by $k = 2$ hops through shared attributes and transfers, and builds a view of that region over a $900\,\mathrm{s}$ window, together with the *global* degree of every attribute in it, so $h(\deg a)$ is the same as in a full pass. The first hop may cross attributes with up to 250 users (so the co-members of a hub are re-examined); later hops cross only attributes with at most 20 users, which stops the region from spreading through hubs. Incremental detectors (hub-aware shared attribute, temporal cycle, smurfing) run on the view; whole-graph detectors (the static Cypher rules and community detection) run on a timer.

D. Ensemble, tracking and scoring (C3)

Candidates from different detectors are merged when their Jaccard overlap is at least 0.5, or when one contains 80% of the other and they differ in size by at most a factor of two. A tracker matches candidates to existing rings across batches; a ring is *confirmed* after two consecutive hits and dissolved after three misses. Confirmed rings are scored by a logistic-regression model over 19 ring features (size, density, which attribute kinds are shared, the hub-adjusted weight, cycle presence and length, burstiness, account ages, internal transfers per member, amount statistics, growth, hub exposure and the number of agreeing detectors), calibrated with isotonic regression [7]. A ring becomes an alert when its calibrated risk reaches the alert threshold θ_a. Each alert carries the per-feature contributions to the score, the subgraph of shared attributes and transfers that formed it, and counterfactuals (for the leading features, the value at which the risk would fall below the threshold, and for hub-aware rings the degree the shared attribute would need before the rule stops firing). The model is trained on seeds disjoint from all evaluation seeds.

IV. Experimental Setup

A. Simulator and scenarios

The simulator is a deterministic discrete-event generator: a scenario file and a seed fix the whole stream, which we verify by a digest of the event sequence recorded with every result. Benign users join and leave, link to devices, cards and IPs drawn from heavy-tailed popularity, and transact; benign hubs (offices, carrier NAT, kiosks, family devices) are generated with configured sizes. Ring types are shared device, card or IP, mixed attribute, money cycle, layered chain and smurfing fan-in/fan-out. Evasion strategies are hub hiding (the ring shares a busy benign hub), low-and-slow activity, identity rotation, decoy edges to benign users, and an adaptive attacker that shifts its strategy mix away from strategies that were flagged. Table I lists the scenarios.

B. Protocol

Seeds are split three ways: scorer training (101–110, test 201–205), threshold selection (validation, 301–303) and evaluation (test, 1001 and up). On the validation seeds we evaluated a grid of alert thresholds $\theta_a \in \{0.2, 0.3, 0.5, 0.7, 0.8, 0.9\}$ and edge thresholds $\theta_e \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$ on the five accuracy scenarios; the grid was widened whenever the best value sat on its edge. The selected values are $\theta_a = 0.8$ and $\theta_e = 0.3$ (mean validation F1 0.72). We report the proposed

TABLE I
SCENARIOS (FROM `configs/scenarios/*.yaml`).

Scenario	Users	Dur. (s)	Ev./s	Rings and evasion
baseline	30	60	10	4 planted, none
high_noise	150	60	50	4 planted + random, decoys
hub_heavy	100	60	25	4 planted, hub hiding
adversarial	120	90	30	5 planted + random, all five
drift	200	90	40	2 planted + random, hub hiding
demo_story	120	90	20	3 planted, hub hiding
scale_sweep	10^3+	120	100	latency benchmark (E4)

method both with these values and with the a-priori values $\theta_a = \theta_e = 0.5$.

Every method in an experiment processes the *identical* event stream for a given scenario and seed (the attacker does not receive feedback in these runs), so differences between methods are paired. We report the mean and 95% t-interval over runs, and test differences with the Wilcoxon signed-rank test on paired runs, with Cohen's d_z as effect size.

C. Metrics

A flagged ring *matches* a true ring when their member sets overlap with Jaccard at least 0.5. Ring precision is the fraction of flagged rings that match a true ring; ring recall is the fraction of true rings matched at some point while active. A run in which nothing is flagged has undefined precision; we count its F1 as 0. Time to detect (TTD) is the time from a ring's activation to its first matching flag. Detection latency is the wall-clock time per micro-batch for the dirty-region expansion, the view and the incremental detectors.

D. Baselines

- **Static Cypher, cap 100 / cap 10.** The original rules: shared device (Q1), shared card (Q2), 3-cycles of transfers (Q3), and shared IP with a degree cap (Q4a/Q4b), run on the whole graph every 5 s of simulated time.
- **Louvain.** Louvain communities [4] of the user graph projected through shared attributes and transfers, kept when dense enough.
- **Connected components.** Weakly connected components of the same projection, no density filter.

All baselines use the same two-hit confirmation as the proposed method.

E. Environment

All runs used one laptop (Intel Core i3-1115G4, 2 cores / 4 threads, 7.7 GB RAM, Windows 11), Python 3.11.9, NumPy 2.2.6, scikit-learn 1.7.2, NetworkX 3.5, with experiments run one at a time.

V. RESULTS

A. RQ1: accuracy against baselines

Table II pools 300 runs (5 scenarios × 10 test seeds × six methods). The proposed method reaches ring F1 0.73 (±0.05), against 0.21 for the static rules with a degree cap

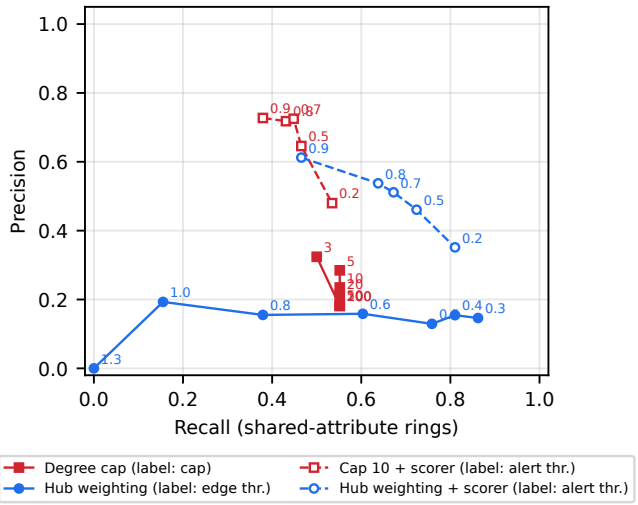


Fig. 2. Precision and recall on shared-attribute rings (micro-averaged over 4 scenarios × 5 test seeds). Solid: detector alone, swept over the degree cap or the edge threshold θ_e. Dashed: with the calibrated scorer, swept over the alert threshold θ_a (cap 10; $\theta_e = 0.3$).

of 10, 0.19 with a cap of 100, 0.29 for Louvain and 0.00 for connected components; each difference is significant on the paired streams ($p < 0.001$). The static rules reach recall 0.61 but precision only 0.14: the shared-attribute rules and the 3-cycle rule fire on many benign groups, and two-hit confirmation does not remove them. Connected components never produce a group within the ring size limits, because benign hubs join almost all users into one component; Louvain splits that component, but its communities rarely coincide with a ring. The thresholds selected on validation seeds improve on the a-priori ones (F1 0.73 vs. 0.68, $p = 0.03$).

Per scenario (Table III), the gap to the static rules is largest on `drift` and `high_noise`, where many benign groups satisfy the rules and their precision collapses, and smallest on the clean `baseline` scenario and on `adversarial`, which is the hardest scenario for every method.

Degree cap vs. hub weighting. Fig. 2 isolates the mechanism this paper replaces. Only shared-attribute and mixed rings count as ground truth here (flags that match only other ring types are ignored), the static rules run without the cycle rule, and the hub-aware detector runs alone. Without the scorer, no degree cap reaches a recall above 0.55: rings that share a busy attribute are invisible to every cap. Hub weighting reaches recall 0.86, but on its own at precision 0.15. With the calibrated scorer on top, hub weighting reaches recall 0.81 (precision 0.35) and its best F1 is 0.58 (alert threshold 0.8); the cap with the scorer stays below recall 0.53 and its best F1 is 0.55. At low recall the cap with the scorer is the more precise of the two (0.72 at recall 0.45). Hub weighting therefore does not dominate the cap everywhere; it extends the frontier to recall levels the cap cannot reach, and the scorer is what makes those levels usable.

TABLE II

RING-LEVEL DETECTION ON 5 SCENARIOS × 10 TEST SEEDS (MEAN ± 95% CI OVER RUNS; A RUN WITH NO FLAGS HAS UNDEFINED PRECISION AND COUNTS AS F1 = 0). TTD: MEDIAN TIME FROM RING ACTIVATION TO FIRST FLAG. LATENCY: MEDIAN PER-BATCH DETECTION TIME ON THE IN-MEMORY STORE; †FULL-GRAPH METHODS RUN ON A 5 S TIMER, NOT PER BATCH, SO THEIR ENTRY IS THE MEDIAN TIME OF ONE FULL-GRAPH PASS. p: PAIRED WILCOXON SIGNED-RANK TEST OF F1 AGAINST THE PROPOSED METHOD ON IDENTICAL STREAMS.

Method	Precision	Recall	F1	TTD (s)	Latency (ms)	p
Static Cypher (cap 100)	0.12 ± 0.03	0.61 ± 0.08	0.19 ± 0.05	9.5	6.0†	<0.001
Static Cypher (cap 10)	0.14 ± 0.04	0.61 ± 0.08	0.21 ± 0.05	9.5	5.9†	<0.001
Louvain communities	0.39 ± 0.09	0.33 ± 0.08	0.29 ± 0.06	18.8	38.4†	<0.001
Connected components	–	0.00 ± 0.00	0.00 ± 0.00	–	11.1†	<0.001
Hub-aware incremental (proposed)	0.76 ± 0.06	0.74 ± 0.06	**0.73 ± 0.05**	9.3	26.6	–
Proposed, a-priori thresholds	0.73 ± 0.06	0.69 ± 0.06	0.68 ± 0.05	9.0	22.7	0.034

TABLE III

RING F1 PER SCENARIO (MEAN ± 95% CI OVER 10 TEST SEEDS).

Method	baseline	high_noise	hub_heavy	adversarial	drift
Static Cypher (cap 100)	0.48 ± 0.07	0.06 ± 0.02	0.24 ± 0.04	0.14 ± 0.04	0.05 ± 0.02
Static Cypher (cap 10)	0.54 ± 0.08	0.07 ± 0.02	0.27 ± 0.05	0.14 ± 0.04	0.05 ± 0.02
Louvain communities	0.44 ± 0.09	0.26 ± 0.19	0.24 ± 0.09	0.14 ± 0.09	0.38 ± 0.16
Connected components	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Hub-aware incremental (proposed)	0.99 ± 0.03	0.62 ± 0.08	0.79 ± 0.08	0.59 ± 0.14	0.67 ± 0.04
Proposed, a-priori thresholds	0.93 ± 0.07	0.66 ± 0.05	0.66 ± 0.12	0.53 ± 0.11	0.62 ± 0.09

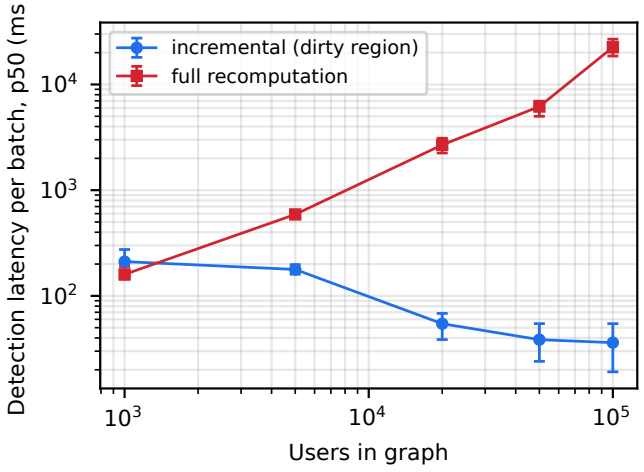


Fig. 3. Median per-batch detection latency as the graph grows (mean over 3 seeds; bars: per-seed minimum and maximum), incremental (dirty region) vs. full recomputation of the same detectors, in-memory store.

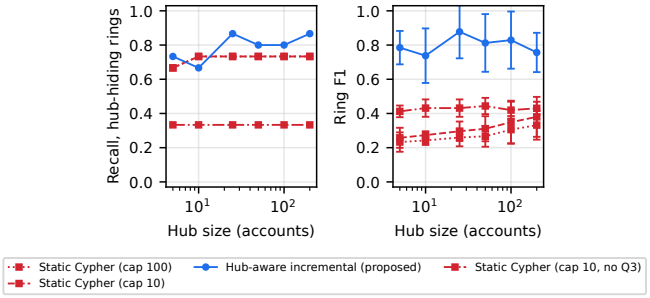


Fig. 4. A shared-IP ring hiding behind an office hub of growing size (`hub_heavy`, 5 test seeds per size). Left: recall of the hub-hiding rings; right: ring F1 (mean, 95% CI).

B. RQ2: latency and scale

Table IV and Fig. 3 grow the user population of the `scale_sweep` scenario at a fixed event rate (100 events/s) and compare incremental detection with recomputing the same detectors on the whole graph for every batch. Full recomputation grows at least linearly with the graph (faster between 50,000 and 100,000 users, where the 7.7 GB test machine was short of memory; we did not separate the two effects). The incremental region stays between a few hundred and two thousand users, because the fan-out limits stop the expansion at hubs; it even shrinks as the graph grows, since this scenario scales the number of devices, cards and IPs with the users (so an ordinary attribute is shared by fewer users) while hub sizes stay fixed. Its latency therefore stays flat: at 100,000 users a batch takes 36.1 ms incrementally against 22,501 ms in full (623×). At 1,000 users the region already covers almost the whole graph, and the incremental mode is slower than full recomputation because of the cost of expanding the region. On the accuracy scenarios, incremental and full recomputation flag exactly the same rings in every run (ablation *no incremental* in Table V), so restricting detection to the dirty region loses nothing there.

On the small accuracy scenarios one static full-graph pass is cheap (5.9 ms) and cheaper than one batch of the proposed detectors (26.6 ms), which build the hub-aware projection and peel it. The proposed detectors run per batch, the static rules only on a timer; their time to detect is similar here (9.3 s vs. 9.5 s) because the timer runs every 5 simulated seconds.

TABLE IV
PER-BATCH DETECTION LATENCY (MEDIAN, MS; MEAN ± 95% CI OVER 3 SEEDS) AND USERS EXAMINED PER BATCH, INCREMENTAL VS FULL RECOMPUTATION, IN-MEMORY STORE.

	Incremental		Full		
Users	Region	ms	Region	ms	Speed-up
1,000	976	211.0 ± 142.9	1015	159.7 ± 64.4	0.8×
5,000	1878	177.8 ± 46.7	5016	589.8 ± 67.3	3.3×
20,000	1324	54.6 ± 37.1	20010	2683.7 ± 1062.0	49.2×
50,000	1110	38.5 ± 38.2	50011	6181.6 ± 2576.4	160.4×
100,000	827	36.1 ± 44.2	100013	22500.5 ± 10278.5	623.0×

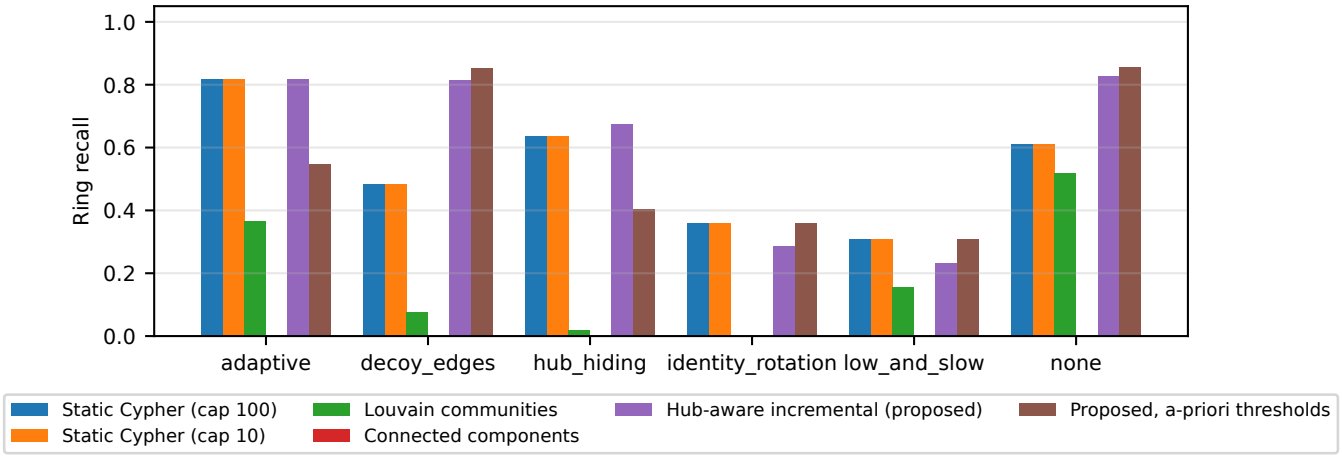


Fig. 5. Ring recall by evasion strategy, pooled over all E1 runs.

C. RQ3: robustness to evasion

Hub size. In Fig. 4 the office hub grows from 5 to 200 accounts. Recall of the hub-hiding rings does not fall with hub size for any method (proposed 71/90, static cap 10 65/90 over all sizes). The static rules do not find these rings through the shared hub: with the 3-cycle rule switched off, cap 10 finds 30/90, because the rings also move money among themselves in short cycles. What separates the methods is precision, and so F1 (0.80 vs. 0.31).

Evasion strategies. Fig. 5 breaks recall down by the strategy each planted or random ring uses. The proposed method's advantage in recall comes mostly from rings without evasion (91/110 vs. 67/110 for cap 10) and rings with decoy edges (22/27 vs. 13/27). On hub-hiding rings the two are close (35/52 vs. 33/52), and identity rotation (4/14) and low-and-slow activity (3/13) defeat every method we tested, presumably because rotation removes the shared attributes the detectors rely on and slow rings accumulate little evidence inside one window.

Adaptive attacker. With feedback switched on, the attacker is told which of its rings were alerted and shifts its strategy mix. Over 10 test seeds it switched strategy 22 times and launched 22 adaptive rings, of which 18 were detected; without feedback, 9 of 11. Adaptation did not lower the detection rate in our simulator, although its strategies are limited to the four we implemented.

D. RQ4: sensitivity

Raising the event rate from 10 to 100 events/s lowers the proposed method's F1 from 0.61 to 0.43 and raises its per-batch latency from 10.1 to 74.5 ms, since each batch touches more of the graph (Fig. 6). Larger planted rings are harder (F1 0.98 at size 3, 0.76 at size 8); a likely cause, which we did not isolate, is the 0.75 edge density the peeling requires. The view window between 60 and 900 s changes F1 little (0.64 vs. 0.67).

E. RQ5: ablations

Table V removes one component at a time (20 paired runs). The calibrated scorer contributes most: without it F1 falls from 0.72 to 0.39 because precision drops to 0.28. Without the hub-aware detector, F1 falls to 0.36 and to 0.00 on `hub_heavy`. Removing the cycle or the smurfing detector costs 0.64 and 0.65. Replacing the hub weight by a constant collapses `hub_heavy` (0.81 to 0.42) but slightly helps `high_noise` (0.54 to 0.68), so over all four scenarios the difference is not significant ($p = 0.161$). Hysteresis and time decay make no significant difference here; without decay F1 is even slightly higher (0.75). The accuracy scenarios last 60–90 s, short against the 300 s half-life, so decay has little to act on here; its value on long streams is untested.

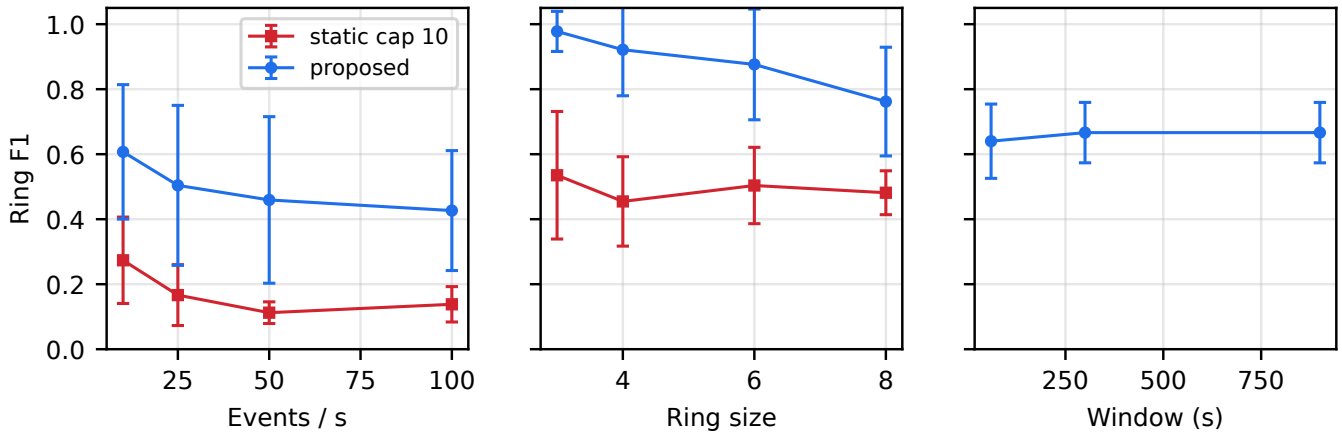


Fig. 6. Ring F1 of the proposed method and of the static rules (cap 10) when the event rate (`adversarial`), the planted ring size (`baseline`) and the view window (`drift`, proposed only) change; 5 test seeds per point.

TABLE V
ABLATIONS ON ADVERSARIAL, HUB_HEAVY, HIGH_NOISE, BASELINE ($\times$ 5 TEST SEEDS). ΔF1: PAIRED MEAN DIFFERENCE TO THE FULL SYSTEM; d_z: PAIRED EFFECT SIZE OF THE FULL SYSTEM OVER THE VARIANT; p: WILCOXON SIGNED-RANK.

Variant	Precision	Recall	F1	ΔF1	d_z	p	Latency (ms)
Full system	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	–	–	–	19.2
w/o hub weighting	0.65 $\pm$ 0.15	0.58 $\pm$ 0.14	0.60 $\pm$ 0.14	-0.12	0.42	0.161	24.0
w/o time decay	0.80 $\pm$ 0.11	0.72 $\pm$ 0.10	0.75 $\pm$ 0.09	+0.02	-0.18	0.277	17.6
w/o incremental scheduling	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	-0.00	–	–	18.4
w/o risk model	0.28 $\pm$ 0.09	0.82 $\pm$ 0.09	0.39 $\pm$ 0.10	-0.33	2.05	$<$0.001	15.1
w/o hysteresis	0.72 $\pm$ 0.12	0.74 $\pm$ 0.10	0.70 $\pm$ 0.10	-0.02	0.37	0.154	22.7
w/o hub-aware detector	0.98 $\pm$ 0.05	0.25 $\pm$ 0.09	0.36 $\pm$ 0.12	-0.36	1.02	0.001	5.3
w/o cycle detector	0.75 $\pm$ 0.14	0.60 $\pm$ 0.11	0.64 $\pm$ 0.11	-0.09	1.02	0.002	15.4
w/o smurfing detector	0.72 $\pm$ 0.14	0.64 $\pm$ 0.15	0.65 $\pm$ 0.14	-0.07	0.93	0.003	19.9

F. Public data

[TBD: AMLworld HI-Small: slice, rows, attempts in slice, account-level precision/recall, attempt recall by typology. Elliptic and AMLSim: evaluated or not, per what could be downloaded.] The public datasets have no device, card or IP links, so only the transaction detectors (temporal cycle and smurfing) run on them and these results do not bear on hub-aware weighting.

VI. THREATS TO VALIDITY

Simulator realism. All main results come from our simulator. Hub sizes, ring behaviour and evasion strategies follow our configuration, not observed fraud. A detector tuned to this generator may do worse on real traffic, and the absolute numbers should not be read as field performance.

Synthetic labels. Ground truth is the set of rings the simulator planted. Benign groups that happen to look like rings count as false positives, and planted rings that never become visible in the stream still count against recall; both directions affect every method equally.

Public data. The public datasets we could obtain carry no device, card or IP links, so they cannot test hub-aware weighting at all; on them only the transaction detectors run (Section V-F). Elliptic and AMLSim were [TBD: evaluated / not evaluated, depending on downloads].

Tuning. Thresholds were selected on validation seeds of the same generator; the test seeds are independent draws but from the same distribution.

Scale. [TBD: state the largest graph measured (100,000 users) and that 10^6 was not feasible on the test machine, if so].

Latency. Latencies are measured on the in-memory store on a laptop; Neo4j adds query and transport cost that we did not include in these numbers.

Baselines. We do not compare with GNN classifiers, which answer a different question (per-node labels rather than groups) and need labelled training graphs.

VII. CONCLUSION

[TBD: Written from the final results.]

REPRODUCIBILITY

Code, scenario files, trained models and the scripts that produce every table and figure are in the project repository; `REPRODUCE.md` lists the exact commands. Each result row records its scenario hash, method configuration hash and event-stream digest.

REFERENCES

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, p. 113303, 2020.

[2] L. Akoglu, H. Tong, and D. Koutra, "Graph based anomaly detection and description: a survey," *Data Mining and Knowledge Discovery*, vol. 29, no. 3, pp. 626–688, 2015.

[3] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 895–904.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[5] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019.

[6] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[7] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002, pp. 694–699.